



Universidad Iberoamericana (UNIBE)

Escuela de Ingeniería TIC

Documento Técnico

Security Smart Services (S.S.S.)

Plataforma de auditoría y monitoreo de seguridad para redes domésticas y pequeñas empresas

Integrantes (Grupo 06)

Pedribel Pión Rijo, 24-0429

Oscar Osnarci Jiménez Peguero, 24-0531

Luca Sita Rincón, 23-0790

Elmer González Otaño, 24-0697

Proyecto Integrador II (TI3631-01-2026-3)

Profesora: Julissa Mateo Abreu

11 de agosto de 2026

1. Introducción y descripción general del proyecto

Security Smart Services (S.S.S.), presentado inicialmente bajo el nombre de trabajo Security Home Services (S.H.S.), es una plataforma web tipo SIEM simplificado para auditar la seguridad de redes domésticas y de pequeñas empresas. El usuario consulta en lenguaje natural el estado de su red ("¿quién está conectado a mi red?", "escanea los puertos del router"), un agente local ejecuta herramientas de red reales (nmap, ping, traceroute) dentro de esa red, los resultados viajan hacia la nube y una inteligencia artificial los interpreta para generar alertas y reportes entendibles para alguien sin formación técnica en seguridad.

El proyecto nació con el nombre S.H.S. durante la fase de levantamiento de requerimientos, orientado al usuario doméstico; durante la ejecución técnica el nombre se ajustó a S.S.S. para reflejar con mayor precisión un enfoque también aplicable a pequeñas y medianas empresas, sin abandonar el caso de uso doméstico original. El sistema está en producción, activo en el dominio securitysmartservices.site.

1.1 Problema identificado

Las redes domésticas y las pequeñas empresas no tienen visibilidad de qué dispositivos están conectados, qué puertos están abiertos ni si hay vulnerabilidades conocidas (CVE) expuestas. Las herramientas tipo SIEM real (Splunk, IBM QRadar, Microsoft Sentinel) están pensadas para empresas grandes con presupuesto y equipo de seguridad dedicado, e inalcanzables para este segmento. Concretamente, el usuario objetivo:

- No sabe quién se conecta a su red ni desde dónde.
- No puede detectar dispositivos no autorizados en su infraestructura.
- No recibe alertas ante eventos de riesgo (accesos fallidos, IPs desconocidas, puertos abiertos).
- Desconoce las vulnerabilidades activas en sus sistemas (CVEs, configuraciones inseguras).
- No cuenta con un historial auditable de la actividad de su red.

1.2 Público objetivo y modelo de negocio

El proyecto identifica dos públicos con el mismo problema de fondo pero distinto nivel de profundidad: personas sin conocimiento técnico que quieren entender qué pasa en su red doméstica, y pequeñas y medianas empresas (5 a 50 empleados, sin personal de TI dedicado) que manejan información sensible sobre infraestructura básica (routers, computadoras, servidores simples). Dentro de estas últimas se identificaron tres perfiles de usuario: el dueño o gerente con conocimiento técnico limitado, el encargado de IT sin

formación formal en ciberseguridad, y el profesional independiente (médico, abogado, contador) que maneja datos confidenciales de sus clientes.

El modelo de negocio es, a la fecha de este documento, una dirección propuesta y no una decisión comercial ya tomada: el producto no se ha lanzado comercialmente. La dirección evaluada es un modelo SaaS por suscripción mensual, con un nivel de acceso gratuito básico y niveles de pago con mayor capacidad de dispositivos monitoreados, análisis más profundo y reportes automáticos periódicos. Como referencia de mercado se tomaron plataformas como Guardz (guardz.com), que también buscan llevar ciberseguridad accesible a pequeñas empresas; la diferenciación de S.S.S. está en centrarse en la visibilidad de lo que ocurre dentro de la propia red del usuario, en vez de únicamente amenazas externas, combinada con una capa de inteligencia artificial que traduce los hallazgos a lenguaje simple.

2. Arquitectura del sistema

S.S.S. está construido sobre un modelo híbrido nube más agente local. Un servicio en la nube no puede alcanzar, por diseño de red, dispositivos que están detrás del router del usuario; auditar una red privada requiere estar dentro de ella. Esta restricción de red real, no una preferencia técnica, es la que obliga a la arquitectura híbrida: un agente que el usuario instala en un equipo de su propia red es la única pieza con permiso para escanear, y se comunica con la nube manteniendo únicamente conexiones salientes (WSS por el puerto 443), sin abrir puertos ni exigir reglas de firewall especiales.

2.1 Componentes

Capa	Componente	Detalle
Nube	Frontend	React 18 + Vite + TypeScript estricto + Tailwind/shadcn-ui, desplegado en Vercel (CDN)
Nube	Backend (agent)	Express.js sobre Node.js, desplegado en Render; expone API REST y streaming SSE
Nube	Relay WebSocket	Servicio dedicado en Fly.io (región Miami), pasarela WSS entre la nube y los agentes locales
Nube	Base de datos	Supabase (PostgreSQL) con Row Level Security y Realtime
Nube	APIs serverless	Funciones en Vercel (api/): sincronización de CVE, KEV de CISA, OWASP, verificación de contraseñas comprometidas
Nube	Autenticación	Clerk (JWT, OAuth con Google/GitHub/Microsoft,

		registro propio)
Nube	Inteligencia artificial	Groq SDK, modelo Llama 3.3 70B
Nube	Email transaccional	Resend, seis plantillas HTML en español
Local	Scanner-agent	Binario Node.js instalado por el usuario (Windows, macOS o Linux), ejecuta nmap/ping/traceroute

2.2 Por qué un relay WebSocket separado del backend

Las conexiones de los agentes locales son de larga duración y con estado: cada agente mantiene una conexión abierta de forma continua. La API REST del backend, en cambio, es de solicitud y respuesta corta. Mezclar ambos tipos de tráfico en el mismo proceso implica que un problema en uno puede afectar al otro. Separarlos en un servicio propio (Fly.io) aísla el riesgo: si el relay tiene un incidente, la API y el dashboard siguen funcionando con normalidad.

2.3 Entornos

Aspecto	Desarrollo	Producción
Frontend	Local (puerto 8080)	Vercel — securitysmartservices.site
Backend (agent)	Local (puerto 3001)	Render
Relay	Local	Fly.io
Base de datos	Supabase (proyecto de desarrollo)	Supabase (proyecto independiente, RLS activo)

3. Decisiones de diseño de software

Durante la fase de diseño del sistema se evaluaron formalmente los patrones de comportamiento Observer y Strategy (catálogo GoF, documentados en Refactoring Guru) como posibles soluciones a dos problemas del dominio: distribuir un mismo evento de seguridad hacia múltiples destinos sin acoplar el emisor a sus consumidores (Observer), y encapsular distintos niveles de análisis de eventos de forma intercambiable (Strategy). Esa evaluación quedó documentada como tickets de trabajo en el tablero del equipo. En la implementación final que hoy está en producción, el problema de fondo que motivó esos patrones se resolvió mediante mecanismos equivalentes, pero distintos en su forma concreta:

- **Distribución de eventos (equivalente a Observer):** Supabase Realtime cumple, en la práctica, el rol que se había diseñado para Observer: el backend escribe un evento (nueva amenaza, nuevo dispositivo, resultado de escaneo) en la base de datos, y cada

suscriptor interesado (el dashboard, el canal de notificaciones) reacciona de forma independiente vía suscripción a esa tabla, sin que el emisor conozca a sus consumidores.

- **Variación de comportamiento (equivalente a Strategy):** en vez de una jerarquía formal de clases de estrategia por plan de suscripción (que nunca llegó a implementarse porque el modelo de negocio por planes no se activó comercialmente), la variación de comportamiento del sistema se resuelve con el control de acceso basado en roles (RBAC): tres roles (admin, normal, guest) sobre nueve secciones del sistema, cada combinación con un nivel de acceso (ninguno, ver, completo), evaluado en cada endpoint mediante un patrón consistente de middleware (requireAuth seguido de requirePermission y de validación de esquema con Zod).

Esta diferencia entre lo diseñado inicialmente y lo construido es una decisión de ingeniería válida y documentada: se evitó introducir una capa de abstracción formal (clases AnalysisStrategy, EventEmitter explícito) para un caso de uso que Realtime y RBAC ya resolvían de forma nativa dentro del stack elegido, priorizando entregar un sistema funcionando en producción sobre una fidelidad literal al patrón de catálogo.

3.1 Principio de menor privilegio, aplicado en cada capa

El sistema aplica defensa en profundidad de forma consistente: el scanner-agent solo permite una lista blanca de comandos de red (nmap, ping, traceroute, arp, dig, nslookup, whois), sanitiza argumentos prohibiendo caracteres de inyección de shell, limita el escaneo a rangos de red privados (192.168.x.x, 10.x.x.x, 172.16-31.x.x), aplica límite de frecuencia (5 escaneos por minuto) y corre como usuario sin privilegios de administrador. La clave de servicio de Supabase (service_role_key) vive únicamente en el backend y el relay; el frontend usa solamente la clave anónima pública.

4. Modelo de datos y seguridad

La base de datos es PostgreSQL gestionado por Supabase, con más de dieciocho migraciones versionadas a la fecha de este documento. Las tablas principales incluyen perfiles y permisos de usuario, métricas de red, dispositivos detectados, amenazas, escaneos de vulnerabilidades, registros de auditoría, reportes generados, configuración de email, y resultados de escaneo, con suscripción en tiempo real (Supabase Realtime) sobre las tablas que alimentan el dashboard en vivo.

4.1 Hallazgo y cierre de una fuga de datos real en producción

Durante una verificación de seguridad se descubrió que el Row Level Security (RLS) de Supabase no protegía a los usuarios tras la migración de autenticación de Supabase Auth a Clerk. Las políticas de RLS existentes estaban escritas como USING

(auth.uid() = user_id), pero con Clerk el frontend nunca enviaba un token de Clerk a Supabase, por lo que auth.uid() quedaba siempre en NULL: cualquiera con la clave anónima pública del proyecto —expuesta por diseño en el bundle del frontend— podía leer los datos de todos los usuarios de la plataforma. El diagnóstico confirmó el escenario contra la base de producción: RLS estaba desactivado y sin políticas efectivas en las tablas de datos.

La solución aplicada fue la integración nativa de Clerk como proveedor de terceros ("third-party auth") de Supabase: el cliente del frontend pasa el token de Clerk al crear la conexión con Supabase, y las políticas de RLS se reescribieron para leer la identidad real del usuario desde el token (auth.jwt()->>'sub') en lugar de auth.uid(). El cambio se aplicó en producción sin interrumpir el servicio, y se verificó RLS activo con políticas efectivas en las dieciséis tablas de datos afectadas. Un filtro de user_id únicamente en el código del cliente no es una frontera de seguridad real, porque con la clave anónima el identificador del usuario lo determina el propio navegador y es falsificable; la protección debe vivir en la base de datos.

4.2 Control de acceso (RBAC)

Rol	Descripción
admin	Acceso completo a todas las secciones y gestión de usuarios
normal	Acceso a la mayoría de las secciones operativas
guest	Solo lectura de dashboard y red

Los nueve módulos cubiertos por RBAC son: dashboard, red, dispositivos, amenazas, vulnerabilidades, logs, análisis con IA, reportes y configuración; cada combinación de rol y sección tiene un nivel de acceso (ninguno, ver, completo).

5. Infraestructura y despliegue

Toda la infraestructura opera sobre planes gratuitos, lo que introduce restricciones reales que el sistema tuvo que absorber en su diseño en lugar de ignorar:

- **Render (backend) hiberna:** el servicio del backend entra en reposo tras 15 minutos sin tráfico; se mitiga con un mecanismo interno de mantenimiento activo más un servicio externo de monitoreo que hace peticiones periódicas de respaldo.
- **Supabase (base de datos) se pausa:** el proyecto se pausa tras aproximadamente siete días sin actividad; se mitiga con un cron interno que ejecuta una consulta real a la base de datos cada tres horas, respaldado por el propio endpoint de salud (/api/health), que también hace ping real a la base de datos en cada llamada.
- **Vercel limita a 12 funciones serverless (plan Hobby):** el plan gratuito de Vercel limita a doce funciones serverless por despliegue; superar ese límite no rompe la

compilación pero sí rechaza el despliegue completo, dejando la producción congelada en la última versión publicada hasta resolverlo. Esta restricción ya se alcanzó en la práctica y se resolvió consolidando dos endpoints relacionados en uno solo.

El propietario único de todos los paneles de infraestructura en la nube (Vercel, Render, Fly.io, Supabase) es Oscar Jiménez Peguero, quien administra despliegues, variables de entorno y secretos; el resto del equipo accede como colaborador. No existe recuperación a un punto en el tiempo (PITR) en el nivel gratuito de la base de datos, por lo que el respaldo se realiza mediante exportaciones lógicas periódicas (pg_dump) almacenadas fuera de la plataforma.

5.1 Cumplimiento y normativa de referencia

El diseño de seguridad del sistema se alinea, sin certificación formal por tratarse de un proyecto académico, con controles reconocidos: ISO/IEC 27001 (gestión de accesos, criptografía, registro y monitoreo, continuidad), NIST SP 800-53 (familias de control de acceso, auditoría, protección de sistemas y comunicaciones, y planificación de contingencia), y la Ley 172-13 de República Dominicana sobre protección de datos personales, reforzada por un modelo en el que la nube nunca accede directamente a la red privada del usuario.

6. Funcionalidades implementadas

El sistema, en su estado actual en producción, incluye:

- Dashboard en tiempo real con métricas de red, amenazas y dispositivos.
- Escaneo de red por lenguaje natural (nmap, ping, traceroute, arp, dig, nslookup, whois), con la IA interpretando la intención del usuario.
- Inventario de dispositivos conectados con enriquecimiento de MAC, fabricante y sistema operativo.
- Detección de amenazas con severidad y estado, y escáner de vulnerabilidades con CVE/CVSS.
- Monitoreo de disponibilidad tipo ping periódico (Pulse) con historial.
- Generación, envío por correo y descarga de reportes en PDF, con puntuación de seguridad.
- Notificaciones en tiempo real (amenazas, vulnerabilidades, nuevos CVE explotados en el catálogo KEV de CISA).
- Gestión de usuarios con RBAC granular sobre nueve secciones y tres niveles de acceso.
- ACi, asistente conversacional de ciberseguridad basado en IA, para interpretar hallazgos en lenguaje simple.

- Geolocalización de IP pública (ciudad, ISP, ASN aproximados) con mapa y verificación de reputación.
- Chequeo rápido sin necesidad de cuenta ni agente: IP pública, filtración de IP local por WebRTC, salud de navegador/conexión, y verificación de contraseñas filtradas contra Have I Been Pwned (hash SHA-1 calculado en el navegador, solo se envían los primeros cinco caracteres).
- Autenticación completa vía Clerk: registro propio, inicio de sesión con Google, GitHub o Microsoft, recuperación de cuenta.
- Instaladores multiplataforma del scanner-agent para Windows, macOS y Linux, con emparejamiento por código y arranque persistente del servicio.

7. Stack tecnológico

Capa	Tecnología
Frontend	React 18 + Vite + TypeScript (modo estricto) + Tailwind CSS + shadcn/ui
Estado y datos	TanStack React Query + React Context + Supabase Realtime
Backend	Express.js sobre Node.js ≥ 18
Base de datos	Supabase (PostgreSQL) con Row Level Security y Realtime
Autenticación	Clerk (JWT, OAuth Google/GitHub/Microsoft)
Relay de comunicaciones	WebSocket dedicado en Fly.io
Inteligencia artificial	Groq SDK — Llama 3.3 70B
Email transaccional	Resend
Scanner de red	nmap, ping, traceroute y utilidades de red vía child process, con lista blanca y sanitización
Hospedaje	Vercel (frontend y APIs serverless), Render (backend), Fly.io (relay)
Control de versiones	Git y GitHub

8. Conclusiones

Security Smart Services integra, en un mismo producto en producción, decisiones de arquitectura obligadas por restricciones reales de red (el modelo híbrido nube más agente local), un modelo de datos relacional con seguridad reforzada a nivel de fila, un incidente de seguridad real diagnosticado y cerrado sin interrumpir el servicio, y una infraestructura desplegada íntegramente sobre niveles gratuitos cuyas limitaciones se diseñaron para absorber en lugar de ignorar. El sistema demuestra que es posible llevar a un público sin conocimiento técnico una herramienta de auditoría de seguridad de red funcional, en producción con dominio propio, sin comprometer los principios de menor privilegio ni de separación de responsabilidades entre sus componentes.

El documento de patrones de diseño elaborado durante la fase de planificación (Observer y Strategy) reflejó correctamente los problemas del dominio a resolver; la implementación final resolvió esos mismos problemas con las herramientas nativas del stack elegido (Supabase Realtime y RBAC), lo cual se documenta aquí como una decisión de ingeniería consciente y no como una desviación del plan original.

Referencias

- Clerk. (2026). Documentación de Clerk. <https://clerk.com/docs>
- Fly.io. (2026). Documentación de Fly.io. <https://fly.io/docs>
- Grupo 06 (Pión Rijo, P., Jiménez Peguero, O., Sita Rincón, L., y González Otaño, E.). (2026, 20 de junio). Aplicación de patrones de diseño: Security Smart Services. Universidad Iberoamericana.
- Grupo 06 (Pión Rijo, P., Jiménez Peguero, O., Sita Rincón, L., y González Otaño, E.). (2026, 20 de junio). Plan de despliegue de infraestructura: S.S.S. Universidad Iberoamericana.
- Grupo 06 (Pión Rijo, P., Jiménez Peguero, O., Sita Rincón, L., y González Otaño, E.). (2026, 22 de mayo). Levantamiento inicial de requerimientos: Security Smart Services. Universidad Iberoamericana.
- Node.js Foundation. (2026). EventEmitter. Documentación de Node.js. <https://nodejs.org/api/events.html>
- OWASP Foundation. (2021). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>
- Refactoring Guru. (2024). Patrones de diseño. <https://refactoring.guru/es/design-patterns>
- Supabase. (2026). Row Level Security. Documentación de Supabase. <https://supabase.com/docs/guides/auth/row-level-security>